LEARN BEING FORWARD

Programming — TypeScript Programming

TypeScript fundamentals & type system

COMPLETE LEARNING GUIDE

What You'll Learn

TypeScript is JavaScript with a structural, static type system layered on top — it compiles down to plain JavaScript and adds no new runtime behavior, but it catches an entire category of bugs (wrong argument types, typos in property names, forgetting a null check) at compile time instead of at runtime in production. This sub-module covers the mental model that makes TypeScript click: structural (not nominal) typing, the basic type annotations for primitives, arrays, and tuples, how type inference frequently removes the need for explicit annotations, union and intersection types for combining possibilities, literal types and const assertions for precise values, the special types any/unknown/never/void and when each is appropriate, and the type narrowing patterns the compiler uses to progressively refine a type inside conditional branches.

1. Structural Typing: Why TypeScript's Type System Feels Different

TypeScript uses structural typing (sometimes called 'duck typing' at the type level): two types are compatible if they have the same shape, regardless of their names or where they were declared — unlike nominal type systems (Java, C#) where compatibility depends on explicit inheritance or interface declarations. This means a plain object literal satisfies an interface simply by having the right properties, with no 'implements' keyword required, which is both extremely convenient for everyday code and a source of occasional surprise when an object has more properties than an interface strictly requires but is still accepted.

TYPESCRIPT

```
interface Point {
  x: number;
  y: number;
}

function printPoint(p: Point) {
  console.log(`(${p.x}, ${p.y})`);
}

// No "implements Point" needed — this object just happens to have the right shape
const location = { x: 10, y: 20, label: "Home" };
printPoint(location); // valid: structurally compatible with Point
```

2. Basic Types: Primitives, Arrays, and Tuples

TypeScript's basic annotations mirror JavaScript's runtime primitives: string, number, boolean, null, and undefined. Arrays are annotated as T[] or Array<T> (both equivalent), and every element must conform to T. Tuples (readonly or mutable, [string, number]) are fixed-length arrays where each position has its own, potentially different, declared type — useful for functions that return exactly two related values, like a [value, error] pair, where plain array typing would lose that positional precision.

- number[] and Array<number> are exactly equivalent syntaxes — pick one and stay consistent within a codebase.
- A tuple's length is fixed at the type level: [string, number] won't accept a third element without an explicit rest element like [string, number, ...boolean[]].

```typescript
let username: string = "ana";
let scores: number[] = [88, 92, 79];
let coordinate: [number, number] = [10, 20];

function divide(a: number, b: number): [number, string | null] {
  if (b === 0) return [0, "Cannot divide by zero"];
  return [a / b, null];
}

const [result, error] = divide(10, 2);
```

3. Type Inference vs. Explicit Annotations

TypeScript's compiler infers types automatically from initial values, return statements, and usage context in the overwhelming majority of cases, which is why idiomatic TypeScript code has far fewer explicit type annotations than beginners expect — let count = 0 is inferred as number without annotation, and a function's return type is inferred from its return statements. Explicit annotations remain valuable at public API boundaries (exported function parameters and return types) where they serve as enforced documentation, and are required when a variable's initial value doesn't give the compiler enough information (an empty array, a variable declared without immediate initialization).

```typescript
let count = 0;             // inferred: number
let name = "Ana";          // inferred: string
const tags = ["a", "b"];   // inferred: string[]

function add(a: number, b: number) {
  return a + b;             // return type inferred as number — no annotation needed
}

let pending: string[];      // annotation required — no initializer to infer from
pending = ["draft"];
```

4. Union and Intersection Types

A union type (A | B) means a value could be either type, and the compiler only allows operations that are valid on both members until the type is narrowed (see below). An intersection type (A & B) means a value must satisfy both types simultaneously — all of A's properties AND all of B's properties — which is most commonly used to combine several smaller object-shape types into one richer one, rather than with primitives (string & number is an impossible type, since no value can be both at once).

- A union of object types gives you only the properties common to ALL members until narrowed; an intersection gives you the properties of ALL members combined.

```typescript
type ID = string | number;

function printId(id: ID) {
  console.log(`ID: ${id}`);
}

type Named = { name: string };
type Aged = { age: number };
type Person = Named & Aged;        // must have BOTH name and age

const p: Person = { name: "Ben", age: 30 };
```

5. Literal Types and const Assertions

A literal type narrows a primitive type down to one exact value — "success" is a type containing only the string "success", not any string. Literal types are most powerful combined into a union to model a closed set of allowed values (a string enum alternative that compiles to nothing extra at runtime). By default, `let status = "success"` widens to the general type string, but `const status = "success"` is inferred as the literal type "success" since const guarantees the value can't change. The `as const` assertion applies that same narrowing to object and array literals, locking every property down to its literal type and marking the structure readonly.

```typescript
type Status = "idle" | "loading" | "success" | "error";

function setStatus(status: Status) { /* ... */ }
setStatus("loading");    // ok
// setStatus("done");    // error: "done" is not assignable to type Status

const config = { env: "production", retries: 3 } as const;
// config.env is the literal type "production", and config is deeply readonly
```

6. any, unknown, never, and void

any disables type checking entirely for that value — it's an escape hatch that should be used sparingly, since it silently defeats the whole point of using TypeScript wherever it appears. unknown is the type-safe alternative for genuinely unknown values (like parsed JSON or external API data): it accepts any value, but the compiler forbids using it for anything until you've narrowed it with a type check first. never represents a value that can genuinely never occur — the return type of a function that always throws, or the type of an unreachable branch in an exhaustive switch — and is useful for making the compiler enforce exhaustiveness checks. void marks a function that doesn't return a meaningful value (though it may still implicitly return undefined).

- Prefer `unknown` over `any` for untyped external data — it forces an explicit type check or assertion before use, which `any` does not.
- A function typed to return `never` that doesn't actually throw or infinite-loop is a compiler-enforced promise you've broken — the compiler will catch it if a code path can return normally.

```typescript
function parseJson(text: string): unknown {
  return JSON.parse(text);
}

const data = parseJson('{"id": 1}');
// data.id would be a compile error — must narrow `unknown` first:
if (typeof data === "object" && data !== null && "id" in data) {
  console.log((data as { id: number }).id);
}

function fail(message: string): never {
  throw new Error(message);
}
```

7. Type Narrowing: typeof, instanceof, and Guards

Inside a conditional branch, TypeScript's control-flow analysis automatically narrows a union type to the specific member that's possible given the check — typeof value === "string" narrows value to string inside that branch, value instanceof SomeClass narrows to that class, and Array.isArray(value) narrows to an array type. For narrowing custom object shapes that aren't classes, a user-defined type guard function (one whose return type is

value is SomeType) lets you encapsulate a more complex check into a reusable, named function the compiler still understands as narrowing.

```typescript
function formatId(id: string | number): string {
  if (typeof id === "string") {
    return id.toUpperCase();   // narrowed to string here
  }
  return id.toFixed(2);        // narrowed to number here
}

interface Cat { meow(): void; }
interface Dog { bark(): void; }

function isCat(animal: Cat | Dog): animal is Cat {
  return "meow" in animal;
}

function makeSound(animal: Cat | Dog) {
  if (isCat(animal)) animal.meow();
  else animal.bark();
}
```

Common Pitfalls

- Reaching for `any` to silence a type error instead of taking a moment to model the actual type or use `unknown` with a narrowing check.
- Treating a union type as if it already had every member's properties available, before narrowing with a typeof/instanceof check.
- Forgetting that TypeScript's types are fully erased at compile time — `typeof someValue === "SomeInterface"` is meaningless; narrowing must use real runtime checks (typeof, instanceof, in, custom guards).
- Annotating every single variable explicitly even when inference already produces the correct, precise type — adding noise without adding safety.
- Expecting `const` alone to deeply freeze an object's properties at the type level without also using `as const`.
- Writing a function typed to return `never` or throw, but leaving a code path that can actually return normally, which the compiler will flag as an error.

Key Takeaways

- TypeScript uses structural typing — type compatibility is based on shape, not declared names or inheritance.
- Let inference do the work for local variables; reserve explicit annotations for function parameters, return types, and ambiguous initial values.

- Union types (A | B) give you only the shared members until narrowed; intersection types (A & B) combine every member's requirements.
- Prefer `unknown` over `any` for untyped data — it forces a narrowing check before use, preserving type safety.
- Narrowing happens through real runtime checks the compiler recognizes: typeof, instanceof, Array.isArray, the `in` operator, and custom `is` type guards.
- TypeScript's types exist only at compile time and are completely erased from the emitted JavaScript — they have zero runtime cost and zero runtime presence.

Learn More

TypeScript Handbook — The Basics
https://www.typescriptlang.org/docs/handbook/2/basic-types.html

TypeScript Handbook — Everyday Types
https://www.typescriptlang.org/docs/handbook/2/everyday-types.html

TypeScript Handbook — Narrowing
https://www.typescriptlang.org/docs/handbook/2/narrowing.html